

Micro-VLA: Scaling In-Context Robotic Manipulation via Observation-Only Videos *

Adrian Boguszewski, Bartosz Czechowski

June 23, 2026

Opening Remarks

Throughout the report, the model that we are discussing is not a VLA. Initially we experimented with a pretrained, 500M parameter VLA - *SmolVLA*, however we quickly realized that it wasn't computationally feasible for us. One epoch of fine-tuning the action head (not even the VLM backbone) took around an hour on a Google Colab T4 GPU, and yielded around 10% success rate on the simplest task we were interested in. We decided to train an order of magnitude smaller decision transformer from scratch (whose architecture will be disclosed in a later section).

1 The Environments

We decided to use *robosuite*, which its authors describe as "a simulation framework powered by the MuJoCo physics engine for robot learning (...) (that) also offers a suite of benchmark environments for reproducible research". The decision was motivated by its easily configurable environments with an option to swap out robot arms to any of the available models.

1.1 Tasks

We chose a subset of three tasks, based on the following criteria: they have an obvious difficulty ordering, they can be seen as a natural extension of one another, and they have accompanying dataset (with an asterisk described in a later section).

1.1.1 Block Lifting

A cube is placed on the tabletop in front of a single robot arm. The robot arm must lift the cube above a certain height. The cube's location is randomized at

*We dropped the "via Observation-Only Videos", as discussed with our lab teacher.

the beginning of each episode.



Figure 1: *Block Lifting* task demonstration

1.1.2 Block Stacking

Two cubes (red and green) are placed on the tabletop in front of a single robot arm. The robot must place the red cube on top of the green cube. The cubes' locations are randomized at the beginning of each episode.



Figure 2: *Block Stacking* task demonstration

1.1.3 Nut Assembly (one square nut variant)

Two colored pegs (one square and one round) are mounted on the tabletop, and a square nut is placed on the table in front of a single robot arm. The robot must fit the square nut onto the square peg. The nut's location is randomized at the beginning of each episode, while the pegs are fixed.

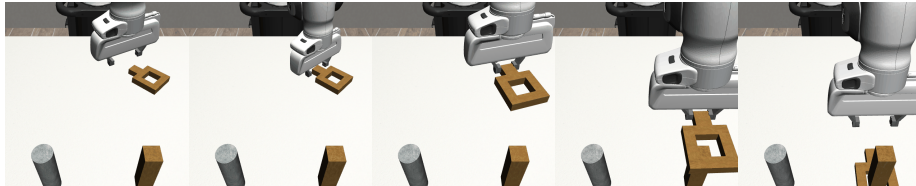


Figure 3: *Nut Assembly* task demonstration

1.2 Robot Arms

Throughout the project we use two robot arms: Panda and IIWA. The first one was chosen, because it was present in the datasets that we had access to

(described later). The second one was selected for its similarity to the Panda - 6 degrees of freedom with similar kinematics and a single-degree-of-freedom gripper. Two distinct arms were necessary to evaluate cross-embodiment generalization.



Figure 4: Panda arm



Figure 5: IIWA arm

2 Training Data

As our training data we mainly used demonstrations sourced from *robomimic*. In its totality, *robomimic* is a more general framework, but we used it solely as a source of human demonstrations and dataset processing scripts.

From *robomimic* we took proficient human and multi-human (divided into "better", "okay" and "worse" operators) demonstrations for block lifting and nut assembly tasks (using the Panda arm), as well as the machine generated trajectories for block lifting (again, using the Panda arm).

What we couldn't find in *robomimic*'s archive, we collected ourselves - block stacking demonstrations (using Panda and IIWA arms), and block lifting demonstrations for the IIWA arm.

We also had to process those datasets to prepare the camera observations, rewards, and feature extractor embeddings of the camera observations for training efficiency (the feature extractor is described in the following section). The original datasets only contained raw trajectories (action, joint and environment state sequences).

All of our training data is publicly available on *HuggingFace* with a comprehensive dataset card that goes into more technical details.

3 Model Architecture

We used a rather standard Decision Transformer architecture - with some implementation details described below.

3.1 Rewards

We made using rewards optional - as when learning from expert trajectories they don't carry much information. But if one decides to use rewards (possibly due to introducing sub-optimal trajectories in the dataset), it's configurable whether to use the dense rewards from the dataset (which grow as the model gets closer to the goal) or to use time based rewards (where we give a reward of -1 at every step, and +100 on success). Then these rewards are used in the return-to-go fashion from the original Decision Transformer paper.

Using dense rewards is sub-optimal as it can be misleading - a model that takes longer to pick up the cube will get a higher total episode return than a model that picks it up faster.

3.2 Observations

As input the model takes a proprioception vector (end-effector position, end-effector rotation, gripper width) and two precomputed Dino3 ViT-Small embeddings of the images from two cameras (eye in hand and robotview). It can be configured whether these should be concatenated into one vector which will be embedded into the latent space of the model, or whether they should be embedded separately and then attended to separately by the model.

We have found that fusing the observations into one gives better results than attending to them separately.

3.3 In-context learning

We also implemented in-context learning - where we also provide the model with a successful trajectory of the task (during training and evaluation).

3.4 Positional Embeddings

We have decided to use learnable positional embeddings as in the original Decision Transformer paper. When using in-context learning we have decided to reset the timestep counter to zero.

Using sinusoidal positional embeddings and not resetting the timestep counters were considered but weren't evaluated.

3.5 Training

We have implemented a standard training loop with MSE loss. We have enabled training on multiple datasets with configurable weighed sampling.

We performed an initial experiment to test whether this architecture was capable of learning the simplest of our tasks - block lifting. The experiment was successful (see Figure 6), however we came across a weird phenomenon (also described in the original *robomimic* paper). The validation loss didn't correlate with model's performance inside the environment. This meant that for every experiment we had to perform validation rollouts inside the environment to

accurately grasp model progress, and that it is worth to train the model much longer than we initially thought.

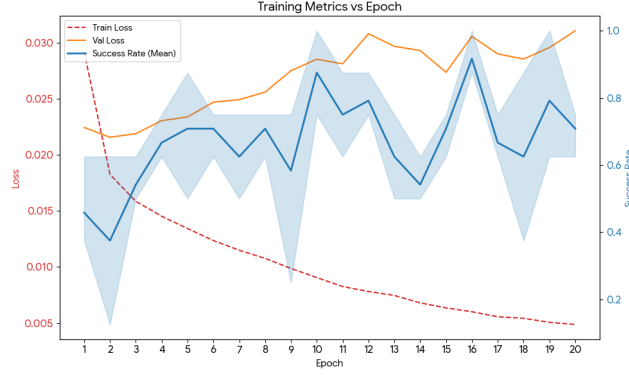


Figure 6: Training and validation losses, as well as success rates plotted against the training epochs on the block lifting task.

4 In Context Learning - Task Generalization

TODO

5 In Context Learning - Cross-embodiment Generalization

TODO

6 Conclusions

The processed datasets can be found on huggingface: <https://huggingface.co/datasets/adrianboguszewski/robomimic> and all the code can be found on github: <https://github.com/czechuuu/micro-vla>.